

GIAO DIỆN (INTERFACE) TRONG JAVA

1. Giới thiệu chung

Trong Lập trình hướng đối tượng (OOP), nếu Lớp trừu tượng (Abstract Class) được ví như một bản thiết kế chưa hoàn chỉnh, thì **Giao diện (Interface)** được xem như một **bản hợp đồng (contract)** hoặc một bản quy ước kỹ thuật.

Interface quy định một tập hợp các hành vi (phương thức) mà bất kỳ lớp nào muốn "ký hợp đồng" (thực thi) với nó đều phải tuân thủ. Interface không quan tâm đến việc lớp đó thực hiện hành vi như thế nào, mà chỉ quan tâm rằng lớp đó có thực hiện hành vi đó hay không.

Tính năng quan trọng nhất của Interface là hỗ trợ cơ chế **Đa kế thừa (Multiple Inheritance)** về mặt hành vi trong Java.

2. Cấu trúc và Cú pháp

Để khai báo một Interface, ta sử dụng từ khóa **interface**.

Cú pháp:

```
public interface TenGiaoDien {  
    // Khai báo các hằng số  
    // Khai báo các phương thức trừu tượng  
}
```

2.1. Thành phần của Interface (Trước Java 8)

Theo quy chuẩn truyền thống, một Interface chỉ bao gồm hai thành phần chính:

1. **Hằng số (Constants):** Mọi biến được khai báo trong Interface đều ngầm định là **public static final**. Tức là chúng là các hằng số toàn cục và không thể thay đổi giá trị.
2. **Phương thức trừu tượng (Abstract Methods):** Mọi phương thức khai báo trong Interface đều ngầm định là **public abstract**. Chúng chỉ có tên, kiểu trả về, tham số và không có phần thân {...}.

Ví dụ minh họa:

```
public interface IThanhToan {  
    // Hằng số (Mặc định là public static final)  
    double THUE_VAT = 0.1;  
  
    // Phương thức trừu tượng (Mặc định là public abstract)  
    void thanhToanOnline(double soTien);  
    void thanhToanTienMat(double soTien);  
}
```

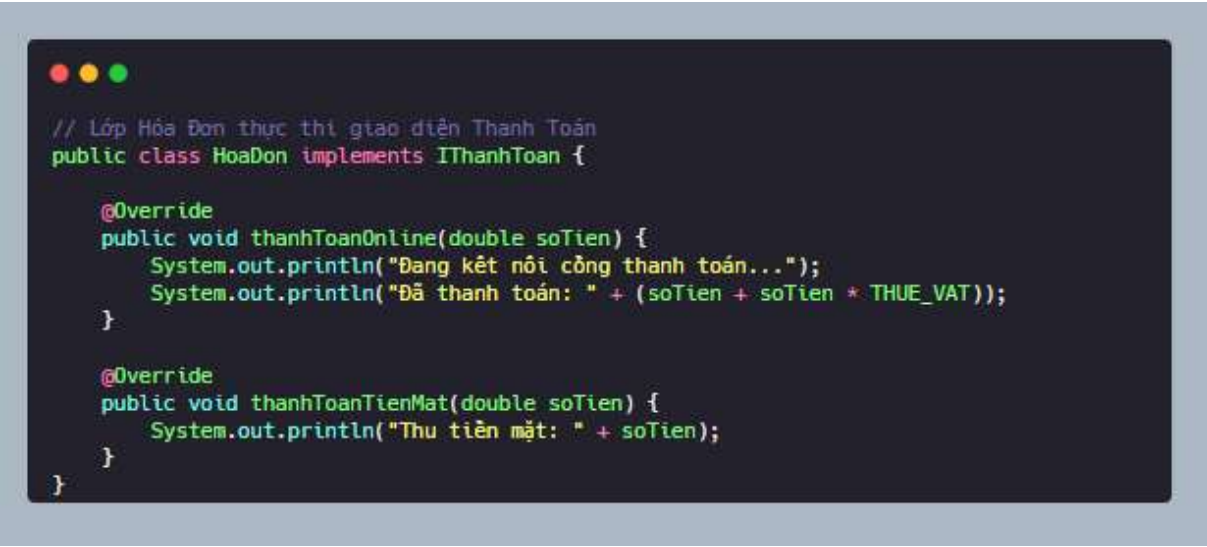
3. Thực thi Interface (Implementing Interface)

Để một lớp sử dụng Interface, ta dùng từ khóa **implements**. Một lớp có thể thực thi **nhiều** Interface cùng lúc (cách nhau bằng dấu phẩy), điều mà kế thừa Class (**extends**) không làm được.

Khi một lớp thực thi Interface, nó có hai lựa chọn:

1. **Triển khai (Ghi đè):** Phải viết mã nguồn chi tiết cho **tất cả** các phương thức trừu tượng có trong Interface.
2. **Trở thành Abstract Class:** Nếu không triển khai hết các phương thức, lớp đó phải được khai báo là **abstract**.

Ví dụ:



```
// Lớp Hóa Đơn thực thi giao diện Thanh Toán
public class HoaDon implements IThanhToan {

    @Override
    public void thanhToanOnline(double soTien) {
        System.out.println("Đang kết nối cổng thanh toán...");
        System.out.println("Đã thanh toán: " + (soTien + soTien * THUE_VAT));
    }

    @Override
    public void thanhToanTienMat(double soTien) {
        System.out.println("Thu tiền mặt: " + soTien);
    }
}
```

4. Các tính năng mới từ Java 8

Trước đây, Interface bị hạn chế vì không thể chứa logic xử lý. Từ phiên bản Java 8, Oracle đã bổ sung hai tính năng quan trọng để tăng tính linh hoạt:

4.1. Phương thức mặc định (Default Method)

Cho phép định nghĩa một phương thức có phần thân ngay trong Interface bằng từ khóa **default**. Các lớp thực thi Interface có thể sử dụng ngay phương thức này mà không cần ghi đè.

- **Mục đích:** Giúp thêm tính năng mới vào Interface mà không làm lỗi các lớp đã thực thi Interface đó từ trước (Backward Compatibility).

4.2. Phương thức tĩnh (Static Method)

Cho phép định nghĩa phương thức tĩnh trong Interface. Phương thức này được gọi trực tiếp thông qua tên Interface, tương tự như phương thức tĩnh của lớp.

Ví dụ Java 8:

```
public interface IGiaoTiep {
    void guiTinNhan();

    // Default method: Có sẵn nội dung
    default void goiDien() {
        System.out.println("Đang thực hiện cuộc gọi...");
    }

    // Static method: Tiện ích
    static void kiemTraKetNoi() {
        System.out.println("Kết nối ổn định.");
    }
}
```

5. Phân biệt Interface và Abstract Class

Đây là nội dung cốt lõi cần nắm vững để thiết kế hệ thống.

Tiêu chí	Interface (Giao diện)	Abstract Class (Lớp trừu tượng)
Từ khóa	interface / implements	abstract class / extends
Đa kế thừa	Một lớp có thể implements nhiều Interface.	Một lớp chỉ có thể extends một Abstract Class.
Hàm khởi tạo	Không có Constructor.	Có Constructor (để lớp con gọi khởi tạo).
Thuộc tính	Chỉ chứa hằng số (public static final).	Có thể chứa biến instance (trạng thái đối tượng).
Phương thức	Chủ yếu là trừu tượng (từ Java 8 có thêm default/static).	Có thể chứa cả phương thức trừu tượng và phương thức thường.

Mục đích	Định nghĩa hành vi ("CAN-DO"). Tạo chuẩn giao tiếp giữa các đối tượng không liên quan.	Xây dựng khung sườn cho các lớp cùng bản chất ("IS-A"). Chia sẻ code chung.
----------	---	---

6. Khi nào nên sử dụng Interface?

Bạn nên ưu tiên sử dụng Interface trong các trường hợp sau:

- Cần đa kế thừa:** Khi bạn muốn một lớp sở hữu hành vi từ nhiều nguồn khác nhau.
- Lập trình hướng giao diện (Programming to Interface):** Để giảm sự phụ thuộc (Loose Coupling). Code nên phụ thuộc vào Interface thay vì phụ thuộc vào lớp cụ thể.
- Tách biệt định nghĩa và cài đặt:** Khi bạn muốn công khai bảng danh sách các chức năng (API) cho người dùng mà không muốn họ quan tâm đến việc hệ thống xử lý bên trong như thế nào.

7. Kết luận

Interface là công cụ mạnh mẽ nhất trong Java để thực hiện tính trừu tượng và đa hình. Nó giúp chuẩn hóa hệ thống, tạo ra sự linh hoạt trong thiết kế và cho phép các thành phần phần mềm tương tác với nhau thông qua các "bản hợp đồng" chặt chẽ thay vì phụ thuộc vào cài đặt cụ thể.